NAME: SOHAN NALE

PRN: 202501070122

BATCH: E21

SUBJECT: ESSENTIAL OF DATA SCIENCE (EDS)

2.1.1.List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".

Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".

Display: Displays the current list of integers. If the list is empty, display "List is empty".

Quit: Exits the program.

The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
list = []

while True:

print("1. Add")

print("2. Remove")

print("3. Display")

print("4. Quit")

m = int(input("Enter choice: "))

if m == 1:

add = int(input("Integer: "))

list.append(add)

print(f"List after adding: {list}")

elif m == 2:

if len(list) == 0:

print("List is empty")

elif len(list) != 0:

remove = int(input("Integer: "))

if remove not in list:

print("Element not found")

else:

list.remove(remove)

print(f"List after removing: {list}")

elif m == 3:

if len(list) == 0:

print("List is empty") else:
print(list)

elif m == 4:

break

else:
```

print("Invalid choice")

OUTPUT:

Average time 0.042 s 42.00 ms
Maximum time 0.063 s 63.00 ms
2 out of 2 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1 33 ms DEBUG

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 11	Integer: 11
List after adding: [11]	List after adding: [11]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 25	Integer: 25
List after adding: [11, 25]	List after adding: [11, 25]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 26	Integer: 26
Element not found	Element not found
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 11	Integer: 11
List after removing: [25]	List after removing: [25]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 25	Integer: 25
List after removing: []	List after removing: []
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 3	Enter choice: 3
List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 8	Enter choice: 8
Invalid choice	Invalid choice

Test case 2 30 ms DEBUG

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 1	Integer: 1
List after adding: [1]	List after adding: [1]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 2	Integer: 2
List after adding: [1, 2]	List after adding: [1, 2]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 3	Integer: 3
List after adding: [1, 2, 3]	List after adding: [1, 2, 3]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 4	Integer: 4
List after adding: [1, 2, 3, 4]	List after adding: [1, 2, 3, 4]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 2	Integer: 2
List after removing: [1, 3, 4]	List after removing: [1, 3, 4]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 6	Integer: 6
Element not found	Element not found
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 3	Enter choice: 3
[1, 3, 4]	[1, 3, 4]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 4	Enter choice: 4

2.1.2.
Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.

Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.

Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.

Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Code:

```
student = { 1: "Amit", 2: "Riya", 3: "Kiran", 4: "Neha", 5: "Arjun", 6: "Pooja", 7: "Rahul", 8: "Sneha", 9: "Vikram", 10: "Anjali" }

 print("Original Dictionary:",student)
```

```
key1 = int(input())

value1 = input()

student.update({key1:value1})

print("After Insertion:",student)

key2 = int(input())

 value2 = input()

if(key2 in student.keys()):

student.update({key2:value2})

print("After Update:",student)

key3 = int(input())

if (key3 in student.keys()):

        student.pop(key3)

         print("After Deletion:",student)

        print("Traversing Dictionary:")

for key,value in student.items():

        print(f"{key} : {value}")
```

OUTPUT:

Average time
0.012 s
12.25 ms

Maximum time
0.017 s
17.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 17 ms DEBUG

Expected output

```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3
Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:
1 : Amit
2 : Riya
3 : Karthik
4 : Neha
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh
```

Actual output

```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3
Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:
1 : Amit
2 : Riya
3 : Karthik
4 : Neha
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh
```

Test case 2 12 ms DEBUG

Expected output

```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
12
Meera
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
6
Divya
After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
1
After Deletion: {2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
Traversing Dictionary:
2 : Riya
3 : Kiran
4 : Neha
5 : Arjun
6 : Divya
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
12 : Meera
```

Actual output

```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
12
Meera
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
6
Divya
After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
1
After Deletion: {2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}
Traversing Dictionary:
2 : Riya
3 : Kiran
4 : Neha
5 : Arjun
6 : Divya
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
12 : Meera
```

2.2.1.
Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

The first line of input contains the array of integers which are separated by space

The last line of input contains the key element to be searched

Output format:

If the element is found, print the index. If the element found multiple times, print the starting index

If the element is not found, print Not found.

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

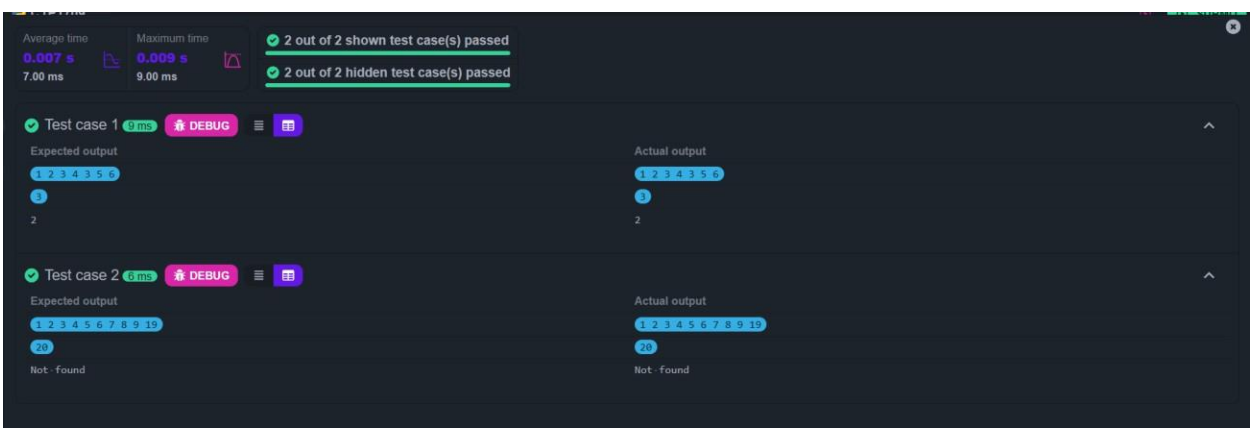
Output:

2

CODE:

```
def linear_search(arr,key):

for i in range(len(arr)):

if arr[i] == key:

return i

return -1
arr = list(map(int,input().split()))

key = int(input())

result = linear_search(arr,key)

if result !=-1:

  print(result)

else:

print("Not found")
```

OUTPUT:



2.2.2._

Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

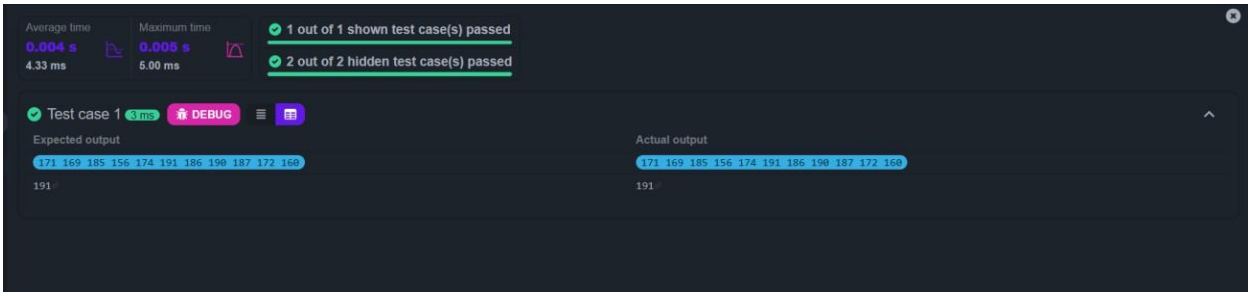
Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

CODE:

```
height = list(map(int,input().split()))
tallest_player = max(height)
print(tallest_player)
```

OUTPUT: